



武汉大学

WUHAN UNIVERSITY

数据结构与程序设计

(Python语言)

DATA STRUCTURE AND PROGRAM DESIGN IN PYTHON

第7讲 函数 (二)



Python课程组



2025年02月



CONTENTS

目 录

- | | | | |
|---|--------|---|----------|
| 1 | 函数式编程 | 5 | 命名空间和作用域 |
| 2 | 递归函数 | 6 | 模块和包 |
| 3 | 嵌套定义函数 | 7 | 实践指导 |
| 4 | 装饰器 | | |



01

函数式编程

函数式编程的概念、Python函数式编程、内置函数map()、lambda表达式（匿名函数）、内置函数filter()、functools模块的reduce()函数



什么是函数式编程

■ 是一种编程范式（编写程序的方法）

✿ 命令式编程

- 编写程序时关注计算机执行的步骤，要告诉计算机先做什么，然后做什么，最后做什么。

✿ 函数式编程

- 更强调程序执行的结果而非执行的过程，倡导利用简单的执行单元让计算结果不断渐进，逐层推导复杂的运算，而不是设计一个复杂的执行过程。
- 在函数式编程中，程序被表示为一系列函数的调用。

✿ 函数式编程的特点

- 虽然函数式编程可能不如面向对象编程和命令式编程那样直观，但它可以帮助开发人员编写更加模块化、低冗余、可读且高效的代码。许多现代编程语言都支持函数式编程，如 Haskell、Erlang、Scala、Clojure 和 F# 等。
- 函数式编程是一种以函数为核心，强调可重用性、无副作用和简洁性的编程范式。



什么是函数式编程

函数式编程的一些关键概念

✿ 高阶函数

- 接受其他函数作为参数或返回函数的函数。

✿ 纯函数

- 返回结果只依赖于它的参数，并且在执行过程中没有副作用的函数。

✿ 惰性求值

- 只在需要时才计算表达式的值，从而提高性能。

✿ 递归

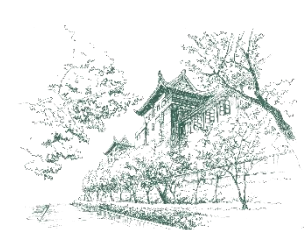
- 通过将问题分解为更小的子问题来解决问题的方法。

✿ 类型推理

- 自动推断函数参数和返回类型的工具。

✿ 模式匹配

- 一种在函数参数中查找匹配项的技术，以确定如何处理不同的输入情况。



什么是函数式编程

✿ 高阶函数

- 高阶函数是指能够接受一个或多个函数作为参数，或者返回一个函数作为结果的函数。
- 常见的高阶函数：
 - **map**: 将一个函数应用于可迭代对象的每个元素，返回一个**map**对象（可以用**list**转换为新的列表）。
 - **filter**: 通过一个函数过滤掉可迭代对象中不满足条件的元素。
 - **reduce**: 通过一个函数将可迭代对象中的元素组合成一个单一的值。
- 将函数作为参数传递给其他函数，可以实现更加通用的算法；而将函数作为返回值，则可以实现惰性执行等功能。

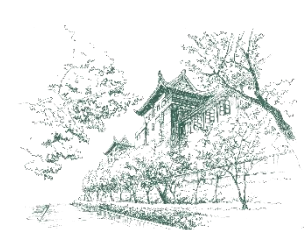
✿ 纯函数

- 纯函数是指在相同的输入条件下，总是返回相同的输出值，并且没有任何副作用的函数。
- 副作用指的是对函数外部环境造成的任何改变，比如修改全局变量、修改传入的参数等。
 - 确定性：给定相同的输入，纯函数总是返回相同的输出。
 - 无副作用：纯函数不会修改外部状态或变量。
 - 可测试性：由于纯函数的输出仅由输入决定，因此它们更易于测试和调试。
- 在函数式编程中，纯函数被广泛使用，因为它们有助于编写清晰、可预测和可维护的代码。



Python函数式编程

- Python不是一种严格意义上的函数式编程语言，但是通过使用**高阶函数、纯函数、匿名函数和内置函数**等也能实现函数式编程。
- 在Python中，一个函数就是一个对象。
 - ✿ 可以把一个函数赋值给变量
 - ✿ 也可以把函数作为参数传入其他函数
 - ✿ 还可以把函数存储在其他数据结构中
 - ✿ 当然也可以把一个函数作为函数的返回值



Python函数式编程



武汉大学
WUHAN UNIVERSITY

■ 示例：计算器模拟程序

✿ 定义纯函数

共定义了7个纯函数，其中有2个高阶函数
(`calc_function`和`calc`)

```
def add(x, y):  
    """加法运算 +"""  
    return x+y  
  
def subtract(x, y):  
    """减法运算 -"""  
    return x-y  
  
def multiply(x, y):  
    """乘法运算 *"""  
    return x*y  
  
def divide(x, y):  
    """除法运算 /"""  
    return x/y
```

首先定义了4个函数，分别完成
两个数的加、减、乘、除运算



Python函数式编程



武汉大学
WUHAN UNIVERSITY

■ 示例：计算器模拟程序

✿ 定义纯函数和高阶函数

函数调用表达式 `calc_function('+')`
的返回值是函数 `add`

```
def calc_function(operator_code): # 高阶函数（返回函数）
    """根据operator_code的值（运算符）返回完成相应运算的函数"""
    functions = [add, subtract, multiply, divide] # 把函数保存在列表中
    return functions["+-*/*".index(operator_code)] # 根据运算符获取函数，并返回函数

def calc(function, x, y): # 高阶函数（函数做参数）
    """function接收函数（可调用对象）参数"""
    return function(x, y)

def calculator(operator_code, x, y):
    """根据operator_code的值（运算符）完成x和y的相应运算"""
    function = calc_function(operator_code) # 把一个函数赋值给变量function
    result = calc(function, x, y) # 函数作为参数传递
    return result
```

针对函数调用 `calc(add, 12, 34)`，
`return` 返回 `add(12, 34)`，即 46

函数调用 `calculator('+', 12, 34)`，
先获得对应的运算函数 `add`，再调用
函数 `calc`，计算 `add(12, 34)` 并返回



Python函数式编程



武汉大学
WUHAN UNIVERSITY

■ 示例：计算器模拟程序

✿ 主程序

```
# 主程序
print(add(12, 34))

add_function = add          # 把函数add赋值给变量add_function
print(add_function(12, 34))  # 该变量可以作为函数名来被调用

print(calc_function('+')(12, 34))  # 返回的函数可以被调用

print(calc(add, 12, 34))          # 函数add做实参

print(calculator('+', 12, 34))    # 充分体现函数式编程的特点
```

5次函数调用的
执行结果相同

程序依次执行的情况如下：

46
46
46
46
46

当需要实现某个计算任务时，调用函数calculator表明需求即可，该函数是对计算的抽象。当然，程序是通过一系列的函数调用完成了计算任务



Python函数式编程



武汉大学
WUHAN UNIVERSITY

- Python的函数式编程能力相较于其他编程语言（如Haskell、Lisp等）略有不足，但它通过**内置函数**、列表推导式、生成器、第三方库和异步编程仍然可以提供足够强大的函数式编程支持。



内置函数map()

■ 内置函数map() 是一个高阶函数，它用于将一个函数作用于可迭代对象（如列表、元组等）的每一个元素，并返回一个新的可迭代对象（**map对象，也称映射对象**）。

✿ 一般用法：

map(function, iterable)

✿ 其中：

- **function** – 第1个参数是函数，可以是内置函数、自定义函数或**lambda**表达式（匿名函数）；
- **Iterable** – 第2个参数是可迭代对象；
- **map**将函数参数应用于可迭代对象中每个元素，并返回一个新的可迭代对象（**map对象**）。



内置函数map()

■ 示例：输入一个三位数，把每一位上的数分别赋值给a,b,c。

```
# map函数示例  
# 此处是输入的数字字符串，如'789'
```

```
x = input('请输入一个三位数： ')  
a,b,c = map(int, x)  
print(a,b,c)
```

```
请输入一个三位数： 789  
7 8 9
```

```
# map函数返回一个map对象  
# 可以使用list()函数将其转换成列表
```

```
print(map(int,x))  
print(list(map(int,x)))
```

```
<map object at 0x000002870B10B100>  
[7, 8, 9]
```

map对象不能通过print()函数显示元素值，可以调用list()函数把它转换为列表对象后输出



内置函数map()



■ 示例：使用map()函数计算一个数据集中每项数据的平方。

```
# 定义函数计算一个数的平方
def square(x):
    return x*x

# 定义一个列表保存一个数据集
values = [1, 2, 3, 4, 5]

# 调用函数map计算数据集中每项数据的平方，保存到map对象
value_square_map = map(square, values)

# 把map对象转换为列表，然后输出
print(list(value_square_map)) # 输出结果是[1, 4, 9, 16, 25]
```

map对象不能通过print()函数显示元素值，可以调用list()函数把它转换为列表对象后输出



内置函数map()

■ 内置函数map()可以有多个可迭代对象参数:

map(function, iterable1, iterable2, ...)

✿ 说明:

- 要求第一个函数可以接受多个参数，其个数与map的可迭代对象参数个数相同；
- map将函数应用于这些可迭代对象的每个元素。

```
def add(x, y):      # 定义计算两个数相加的函数
    return x+y

# 定义两个向量
vector1 = [1, 2, 3, 4, 5]
vector2 = [11, 22, 33, 44, 55]

# 调用函数map计算两个向量相加
vector3 = map(add, vector1, vector2)

# 把map对象转换为列表，然后输出
print(list(vector3)) # 输出结果是[12, 24, 36, 48, 60]
```



lambda表达式（匿名函数）

■ 语法格式：

lambda 若干变量名: 表达式

✱ 用来声明匿名函数（没有名字的临时使用的小函数）

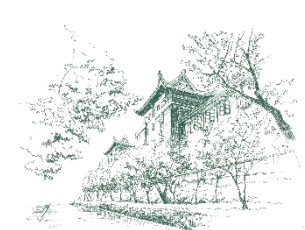
- 若干变量名是函数的形参（用逗号隔开），表达式的计算结果是函数的返回值。
- **lambda**表达式只可以包含一个表达式，不能包含复合语句，但在表达式中可以调用其他函数。

如果一个函数有一个返回值，并且只有一句代码，可以使用**lambda**简化。尤其适合需要一个函数作为另一个函数参数的场合。

```
lambda x:-x                # 定义了一个匿名函数，后续无法被引用

print((lambda x:-x)(123))   # 这样也只能被使用一次，输出-123

square = lambda x:x*x       # 用一个变量来引用lambda表达式
print(square(12))           # 把变量当作函数来调用，输出12的平方144
print(square(3)+square(4))  # 多次调用，输出3和4的平方和25
```



lambda表达式 (匿名函数)

lambda表达式的应用

✿ lambda表达式主要是作为一个简单操作与内置函数map、filter和sorted等配合使用，作为这些内置函数的函数参数或者键函数。

```
# 定义一个变量来引用lambda表达式
```

```
square = lambda x: x*x
```

```
# 定义一个列表保存一个数据集
```

```
values = [1, 2, 3, 4, 5]
```

匿名函数先赋值给square变量，
然后做map函数调用的第一个实参

```
# 调用函数map计算数据集中每项数据的平方，保存到map对象
```

```
value_square_map = map(square, values)
```

```
# 把map对象转换为列表，然后输出
```

```
print(list(value_square_map)) # 输出结果是[1, 4, 9, 16, 25]
```



lambda表达式 (匿名函数)



lambda表达式的应用

示例

```
# 定义两个向量
vector1 = [1, 2, 3, 4, 5]
vector2 = [11, 22, 33, 44, 55]

# 调用函数map计算两个向量相加, lambda表达式直接做实参
vector3 = map(lambda x,y: x+y, vector1, vector2)

# 把map对象转换为列表, 然后输出
print(list(vector3))  # 输出结果是[12, 24, 36, 48, 60]
```

匿名函数有x, y两个参数, map函数有vector1, vector2 两个可迭代对象参数, 两者个数相同



lambda表达式 (匿名函数)

lambda表达式的应用

✿ lambda表达式作为内置函数sorted()的键函数，用来自定义排序规则。

```
# 保存5位学生的姓名和3门课成绩
```

```
score_sheet = [  
    ['李华', 85, 87, 80],  
    ['陈琪', 93, 91, 88],  
    ['余涛', 88, 90, 87],  
    ['郑燕', 82, 78, 80],  
    ['金浩', 91, 88, 85]]
```

```
# 按学生的成绩总分从高到低的顺序排序
```

```
ordered_sheet=sorted(score_sheet,reverse=True,key=lambda student:sum(student[1:]))
```

```
# 显示排序结果
```

```
print(ordered_sheet)
```

```
 [['陈琪', 93, 91, 88], ['余涛', 88, 90, 87], ['金浩', 91, 88, 85], ['李华', 85, 87, 80], ['郑燕', 82, 78, 80]]
```

执行函数sorted时，关键字参数key引用lambda表达式，该表达式的参数student是列表score_sheet的每一个元素，即保存一位学生的姓名和3门课程成绩的列表，表达式的返回值sum(student[1:])是一位学生的3门课程成绩的总分，因此排序规则是比较学生的3门课程成绩的总分

列表对象的成员函数.sort()与内置函数sorted() 都有关键字参数key，都可以使用lambda表达式自定义排序规则，含义相同。



lambda表达式（匿名函数）

lambda表达式的应用

✿ lambda表达式作为内置函数`max()`、`min()`的键函数，用来自定义找最大值或最小值的规则。

```
# 利用max函数从成绩表中找出总分最高的学生
```

```
highest_score_student = max(score_sheet, key=lambda x: sum(x[1:]))
```

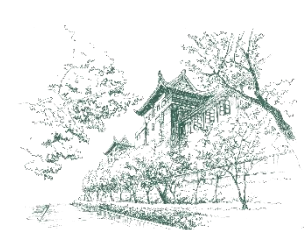
```
# 显示总分最高的学生信息
```

```
print(highest_score_student)
```

执行函数`max`时，关键字参数`key`引用`lambda`表达式，该表达式的参数`x`是列表`score_sheet`的每一个元素，即保存一位学生的姓名和3门课程成绩的列表，表达式的返回值`sum(x[1:])`是一位学生的3门课程成绩的总分，因此找最大值的规则是比较学生的3门课程成绩的总分

```
['陈琪', 93, 91, 88]
```

lambda表达式的变量名，可以写成`student`（见名知意），也可以写成`x`，或者是其他名称，程序执行结果相同。

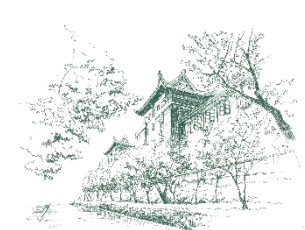


lambda表达式 (匿名函数)

更多lambda表达式应用示例

```
>>> L = [(lambda x: x**2),  
          (lambda x: x**3),  
          (lambda x: x**4)]  
>>> print(L[0](2), L[1](2), L[2](2)) # 列表L的三个元素L[0]、L[1]、L[2]是lambda表达式  
4 8 16  
>>> D = {'f1': (lambda: 2+3),  
          'f2': (lambda: 2*3),  
          'f3': (lambda: 2**3)}  
>>> print(D['f1'](), D['f2'](), D['f3']()) # D['f1']等是lambda表达式 (无变量)  
5 6 8  
>>> lst = [1, 2, 3, 4, 5]  
>>> print(list(map(lambda x: x+10, lst))) # 模拟向量运算  
[11, 12, 13, 14, 15]  
>>> lst  
[1, 2, 3, 4, 5]
```

map() 函数返回一个新的map对象，可迭代对象lst列表的元素值没有改变



lambda表达式 (匿名函数)



更多lambda表达式应用示例

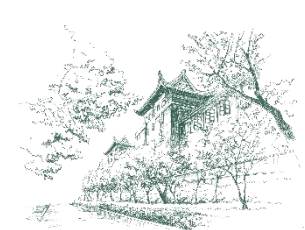
```
>>> data = list(range(20))          # 创建列表
>>> data
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

>>> import random
>>> random.shuffle(data)            # 打乱顺序 (洗牌)
>>> data
[4, 3, 11, 13, 12, 15, 9, 2, 10, 6, 19, 18, 14, 8, 0, 7, 5, 17, 1, 16]

>>> data.sort(key=lambda x: x)      # 和不指定规则效果一样
>>> data
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

>>> data.sort(key=lambda x: len(str(x)))  # 按转换成字符串以后的长度排序
>>> data
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

>>> data.sort(key=lambda x: len(str(x)), reverse=True)  # 降序排序
>>> data
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



lambda表达式（匿名函数）



更多lambda表达式应用示例

```
import random
x=[random.randint(1,10) for j in range(5)] for i in range(5)]
    # 使用列表推导式创建列表（包含5个子列表的列表）
    # 每个子列表中包含5个[1,10]数据区间的随机整数

for item in x:
    print(item)

y = sorted(x, key=lambda item: (item[1], item[4]))
    # 按子列表中第2个元素升序、第5个元素升序排序

print()
for item in y:
    print(item)
```

```
[1, 2, 5, 5, 6]
[6, 6, 8, 6, 5]
[3, 4, 2, 4, 5]
[2, 3, 10, 4, 1]
[4, 4, 10, 5, 4]

[1, 2, 5, 5, 6]
[2, 3, 10, 4, 1]
[4, 4, 10, 5, 4]
[3, 4, 2, 4, 5]
[6, 6, 8, 6, 5]
```

执行函数sorted时，关键字参数key引用lambda表达式，该表达式的参数item是列表x的每一个元素，即保存5个随机整数的子列表，表达式返回的元组(item[1], item[4])用作排序的关键字（或称为排序键）。因此排序规则是比较子列表中下标1的元素和下标4的元素（即第2个和第5个元素）。排序时会首先比较item[1]（第一关键字），如果item[1]相同，则会比较item[4]（第二关键字），以此类推

内置函数filter()



■ 内置函数filter() 是一个高阶函数，用于过滤序列。filter()函数根据判断函数对可迭代对象的元素进行逐一筛选，返回一个新的可迭代对象（filter对象），其中包含所有满足条件的元素。

✿ 一般用法： filter(function, iterable)

- function – 第1个参数是判断函数，判断可迭代对象中的元素是否满足条件。
- iterable – 第2个参数是可迭代对象。

示例：用filter()函数找出平均分大于或等于85的学生

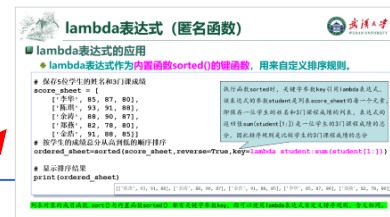
```
excellent_stus = filter(lambda stu:sum(stu[1:])>=85*3,score_sheet)
```

显示满足条件的学生

```
print(list(excellent_stus))
```

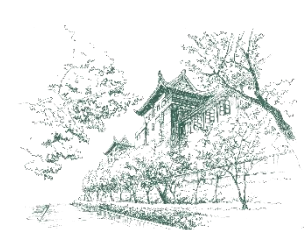
输出结果为：

```
[['陈琪', 93, 91, 88], ['余涛', 88, 90, 87], ['金浩', 91, 88, 85]]
```



执行filter()函数时，lambda表达式的参数stu是列表score_sheet的每一个元素，即保存一位学生的姓名和3门课程成绩的子列表，表达式的返回值是sum(stu[1:])>=85*3，因此，过滤条件是学生的3门课程成绩的总分>=85*3

filter()函数与map()类似，返回值是一个filter对象（不是列表），要用list()函数转换为列表后，再用print()输出其值。



标准库functools模块的reduce()函数



■ **reduce()函数用于对一个可迭代对象（如列表、元组等）进行累积操作**

✿ **一般用法：** `reduce(function, iterable)`

✿ **其中：**

- **function** – 第1个参数是一个二元函数（接收两个参数的函数）。
- **iterable** – 第2个参数是可迭代对象。
- 将**function**函数依次应用于可迭代对象的元素，将前两个元素的结果与下一个元素作为参数传递给二元函数，直到遍历完整个可迭代对象，最后返回一个累积的结果。

```
# 示例：利用函数reduce计算一组数的乘积
from functools import reduce # 先导入functools模块中的reduce
# 有一组数
numbers = [1, 2, 3, 4, 5]
# 计算它们的乘积
result = reduce(lambda x, y: x*y, numbers)
# 显示乘积，输出：120
print(result)
```

执行reduce()函数时，lambda表达式的参数x,y先取列表numbers的第1、2个元素，返回值即乘积1*2=2，乘积2和第3个元素再作为参数，返回值即2*3=6，以此类推，因此，累积结果是((((1*2)*3)*4)*5)



02

递归函数



递归函数

■ 递归函数是指一个函数在执行过程中直接或间接调用自身。

- ✱ 实现递归函数非常简单，只需在函数体内部调用函数本身即可。
- ✱ 在解决具有重复结构的问题时，使用递归函数可以使代码更加简洁和易于理解。

示例：定义一个简单的递归函数，计算斐波那契 (Fibonacci) 数列的第n项，其中第0项是0，第1项是1

```
def fib(n):  
    """用递归方法计算斐波拉契数列的第n项"""  
    if n == 0 or n == 1: # 基本情形（递归基）  
        return n  
    else:  
        return fib(n - 1) + fib(n - 2) # 如果n>1，函数调用自己
```

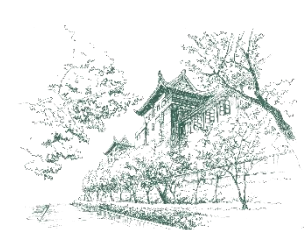
计算斐波拉契数列的第10项，输出结果是55

```
print(fib(10))
```

递归公式：

$$\text{fib}(n) = \begin{cases} 0 \text{ 或 } 1 & (n=0 \text{ 或 } 1) \text{ 基本情形} \\ \text{fib}(n-1) + \text{fib}(n-2) & (n>1) \text{ 简化问题} \end{cases}$$

斐波那契数列：0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368...



递归函数



■ 定义和使用递归函数时，需要注意以下几点：

✿ 递归基（基本情形）

- 递归基是指在什么条件下函数不再调用自身，而是返回一个值或者完成一个简单的操作。
- 在斐波那契数列的例子中，递归基是当 n 等于0或1时，函数返回 n 。

✿ 递归条件

- 递归条件是指在什么情况下函数会调用自身。
- 斐波那契数列函数的递归条件：当 n 大于1时函数会调用自身，计算 $\text{fib}(n-1)$ 和 $\text{fib}(n-2)$ 的和。

✿ 递归终止

- 递归终止是指在什么情况下函数会停止调用自身。
- 斐波那契数列函数的递归终止是通过判断 n 是否等于0或1来实现的。



递归函数



武汉大学
WUHAN UNIVERSITY

■ 示例：汉诺塔问题。

- ✿ 假设有三个塔座，分别命名为X，Y和Z。
- ✿ 设塔座X上有n个圆盘，从上往下按从小到大顺序叠放，编号是1、2、3、...、n。
- ✿ 现在要求将塔座X上的n个圆盘移到塔座Z，移动时必须遵循以下规则：
 - 每次只能移动一个圆盘；
 - 圆盘可以放在X，Y和Z中的任一塔座上；
 - 移动过程始终保持大盘在下、小盘在上的叠放方式。





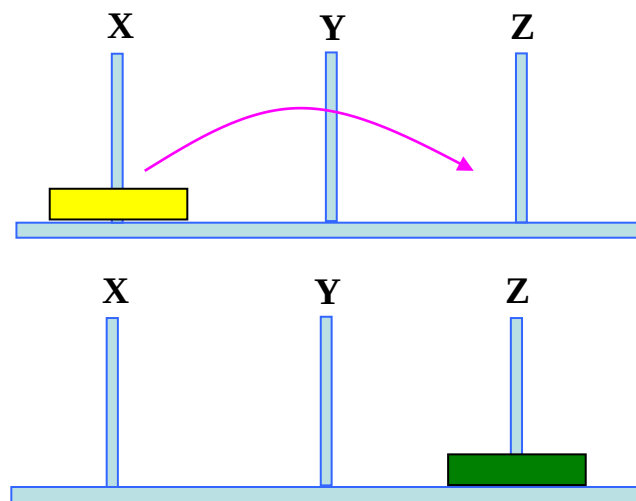
递归函数



■ 示例：汉诺塔问题。

✱ 分析，当 $n=1$ 时：

➤ 直接将圆盘1从塔座X移到塔座Z。



基本情形



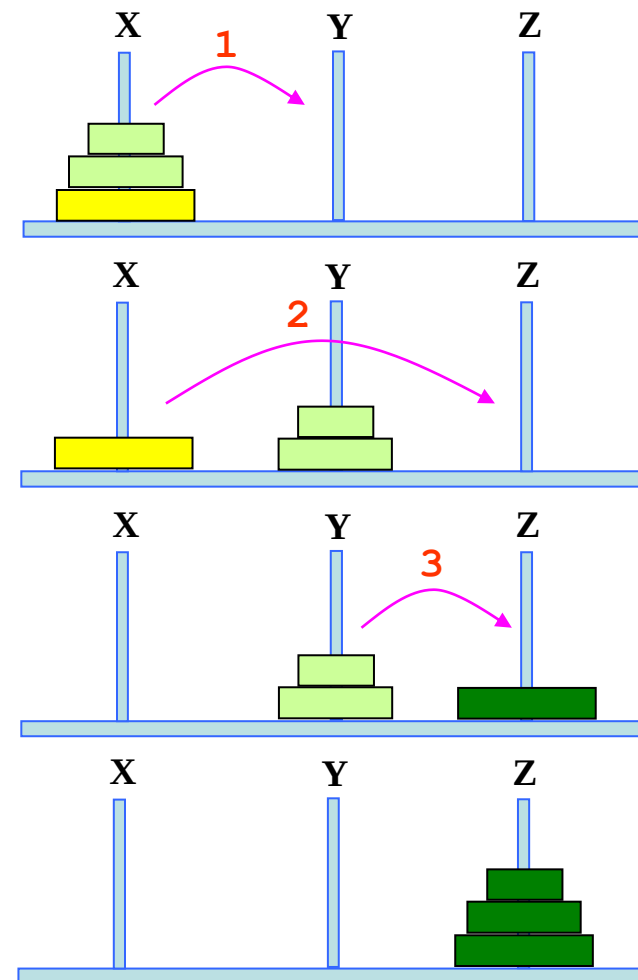
递归函数

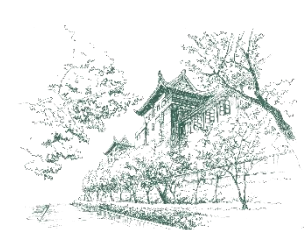


■ 示例：汉诺塔问题。

✱ 分析，当 $n > 1$ 时：

1. 利用塔 z 为辅助塔，先把塔 x 上第 $1 \sim n-1$ 个盘移至塔 y 上；
(—与原问题类似)
2. 剩下的最大圆盘 n 直接从塔 x 移到塔 z ；
3. 利用塔 x 为辅助塔，把塔 y 上的 $n-1$ 个圆盘至塔 z 。
(—与原问题类似)





递归函数



■ 示例：汉诺塔问题。

✦ 设计

➤ move 函数：移动一个盘

- 把编号为 d_i 的盘 从 s 塔移到 d 塔

```
move(di, s, d)
```

➤ hanoi 函数：移动 n 个盘的汉诺塔问题

- 把 n 个盘从 x 塔移到 z 塔， y 塔作为辅助塔

```
hanoi(n, x, y, z)
```



递归函数



■ 示例：汉诺塔问题。

✿ 实现

```
# 定义移动圆盘的函数
def move(di, s, d):
    """把编号为di的圆盘从塔s移到塔d上"""
    print(f"{di}\t{s}-->{d}")
```

定义求解汉诺塔问题的递归函数

```
def hanoi(n, x, y, z):
    """把塔x上的n个圆盘移动到塔z，塔y是辅助塔"""
    if n == 1: # 基本情形：只有1个圆盘需要移动，便直接移动
        move(n, x, z)
    else: # 递归：分三步求解
        hanoi(n - 1, x, z, y)
        move(n, x, z)
        hanoi(n - 1, y, x, z)
```

调用递归函数，求解3个圆盘汉诺塔问题

```
hanoi(3, 'X', 'Y', 'Z')
```

虽然递归函数可以使代码更加简洁，但在某些情况下，它有可能导致栈空间的溢出。因此，在使用递归函数时，需要权衡代码的简洁性和程序的资源消耗

执行程序得到下面的结果：

```
1   X-->Z
2   X-->Y
1   Z-->Y
3   X-->Z
1   Y-->X
2   Y-->Z
1   X-->Z
```



递归函数



更多示例：用递归的方式来反转字符串。

分析与设计

- 将一个字符串视为递归对象，可以把字符串划分为第一个字符和“所有其他字符”；
- 如果反转“所有其他字符”，然后将第一个字符放在反转后的“所有其他字符”之后，就反转了整个字符串。

Hello

olleH

```
def reverse_string(s):  
    if len(s) <= 1:  
        return s  
    return reverse_string(s[1:]) + s[0]
```

```
print("Hello")  
print(reverse_string("Hello"))
```

递归开始回溯时，每一层函数调用都会接收到来自下一层的返回值，并据此构建自己的返回值：

第5层reverse_string("o")返回"o"

第4层reverse_string("lo")返回"o"+"l"，即"ol"

第3层reverse_string("llo")返回"ol"+"l"，即"oll"

第2层reverse_string("ello")返回"oll"+"e"，即"olle"

第1层reverse_string("Hello")返回"olle"+"H"，即"olleH"



递归函数



更多示例：用递归法对整数进行质因数分解。

$$100 = 2*2*5*5$$

$$12129042 = 2*3*2021507$$

$$88601477 = 88601477$$

```
from random import randint
```

```
def factors(num, fac=[]):          # fac的默认值是可变列表
    for i in range(2, int(num**0.5)+1): # 每次都从2开始查找因数
        if num%i == 0:             # 找到一个因数
            fac.append(i)
            factors(num//i, fac)    # 递归调用对商继续分解，重复这个过程
            break                  # 直接跳出for，不执行else，注意，这个break非常重要
    else:
        fac.append(num)            # 没找到因子，num本身是质数不可分解，num是自己的因子添加到列表

factor_list = []
n = randint(2, 10**8)
factors(n, factor_list)
result = '*'.join(map(str, factor_list))
if n == eval(result):
    print(f'{n} = {result}')
```



03

嵌套定义函数



嵌套定义函数

- Python允许**嵌套定义函数**，即在一个函数内部定义另外一个函数。
- 嵌套定义函数可以让代码更加模块化和易于理解。

```
# 定义自己的映射函数
```

```
def my_map(iterable, op, value):  
    if op not in '+-*/':  
        return 'Error operator'
```

```
# 定义内嵌函数，item是通过map函数从iterable参数中逐个获取的
```

```
def nested(item):  
    return eval(str(item)+op+str(value))
```

函数内部定义的内嵌函数属于自己私有的函数，
主要完成函数内部提取的公共操作或辅助功能

```
# 内嵌函数做实参
```

```
return map(nested, iterable)
```

```
# 调用my_map函数，不需要关心其内部实现
```

```
print(list(my_map(range(5), '+', 5)))  
print(list(my_map(range(5), '-', 5)))
```

```
[5, 6, 7, 8, 9]  
[-5, -4, -3, -2, -1]
```




武汉大学
WUHAN UNIVERSITY

04

装饰器



装饰器 (decorator)

■ 装饰器是Python中一种非常有用的功能，它可以在不修改函数代码的基础上，为函数增加新的功能。

- ✿ 装饰器本质上是一个高阶函数，它接受一个函数作为参数，并返回一个经过增强或修改后的函数。
- ✿ 定义函数时，函数前面可以添加一个或多个装饰器，@decorator表示一个装饰器，每个装饰器有自己的名字。
- ✿ 带装饰器的函数定义形式如下

@装饰器1

@装饰器2

@...

def 函数名(参数表):

函数体

带装饰器的函数定义示例

```
@decor  
def func():  
    pass
```

等价于以下代码

```
func = decor(func)
```



装饰器 (decorator)



■ 一个简单的装饰器示例程序。

```
# 定义装饰器 (函数)
def my_decorator(function):
    # 定义内嵌的包装器 (函数)
    def wrapper():
        print("在函数执行之前做一些操作")
        function() # 调用原函数
        print("在函数执行之后做一些操作")

    return wrapper

@my_decorator # 添加装饰器声明
def say_hello():
    print("Hello!")

# 调用say_hello函数
say_hello()
```

在这个例程中，函数my_decorator是一个装饰器，它接受一个函数作为参数；在my_decorator内部定义了一个新的函数wrapper，这个函数可以在调用原始函数function之前和之后分别执行一些操作；最后my_decorator返回这个新的wrapper函数，该函数对原函数的功能进行了增强。

在定义函数say_hello时，在其头部前面添加了装饰器my_decorator声明，那么后续调用say_hello时，实际上是调用my_decorator(say_hello)返回的函数。即say_hello()被转换为my_decorator(say_hello)()

执行程序得到下面的结果：

```
在函数执行之前做一些操作
Hello!
在函数执行之后做一些操作
```




装饰器 (decorator)



武汉大学
WUHAN UNIVERSITY

仿写一个简单的装饰器示例程序。

```
# 定义装饰器 (函数)
def my_decorator(func):
    def wrapper():
        print("💎 魔法咒语启动, 准备变身! 💎")
        func()
        print("🌈 魔法完成, 成功变身! 🌈")
    return wrapper

@my_decorator
def hero_action():
    print("英雄出击, 拯救世界! ")

hero_action()
```

执行程序得到下面的结果:

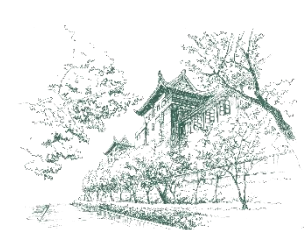
```
💎 魔法咒语启动, 准备变身! 💎
英雄出击, 拯救世界!
🌈 魔法完成, 成功变身! 🌈
```

试一试, 仿写一个简单的函数装饰器!



05

命名空间和作用域



■ 命名空间

- ✱ 命名空间是一个用于存储变量名称（以及函数名和类名）和其对应对象的系统，它确保名称的唯一性。
- ✱ 命名空间可看作是一个字典，其中变量名是字典的键，对应的对象是字典的值。

■ Python中有三种主要的命名空间：

✱ 内置命名空间

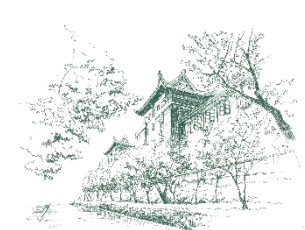
- 它包含了Python内置的函数和类型，例如print和str。

✱ 全局命名空间

- 它包含了在整个程序中定义的全局变量和函数。

✱ 局部命名空间

- 它包含了在函数内部定义的局部变量和函数。



作用域



■ 作用域是指在程序中访问命名空间中变量的有效范围，即可以引用该变量的区域。在Python中，有四种作用域：

✿ 内置作用域

➤ 它包含了内置命名空间中的变量，可以在程序的任何地方访问。

✿ 全局作用域

➤ 它包含了全局命名空间中的变量，可以在程序中的任何地方访问。

✿ 局部作用域

➤ 它包含了局部命名空间中的变量，只能在函数内部访问。

✿ 非局部作用域

➤ 它允许在嵌套函数中访问外部函数的变量。



作用域



- 当在程序中引用一个变量时，Python解释器会按照以下顺序搜索命名空间：
 - ✿ 局部命名空间 -> 全局命名空间 -> 内置命名空间
 - ✿ 不同作用域内的变量名可以相同，互不影响。
- 通过理解Python中命名空间和作用域的基本概念，可以更好地组织和管理变量，避免命名冲突，并且能够正确地访问和使用变量。



局部变量

- ✿ 局部变量是指在函数内部定义的普通变量（包括形参），只在函数内部起作用。
- ✿ 当函数执行结束后，局部变量自动删除，不能再使用。
- ✿ 局部变量的引用比全局变量速度快，应优先考虑使用。
- ✿ 不同函数中可以有同名的局部变量，互不干扰。

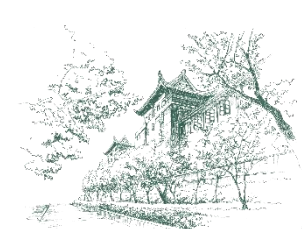
```
def f1():  
    x=5  
    y=6  
    print(x+y)          # f1函数中的y和f2函数中的y互不干扰
```

```
def f2():  
    y=1  
    print(x+y)          # 出错！不能引用f1函数中的x
```

```
f1()
```

```
f2()
```

NameError: name 'x' is not defined



作用域



全局变量

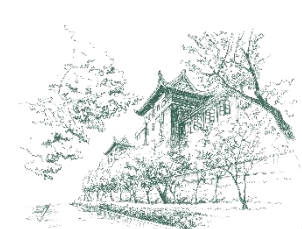
- ✱ 全局变量是指在函数和类定义之外声明的变量。
- ✱ 全局变量的作用域从定义的位置起，直到文件结束。
 - 如果在函数中定义的局部变量与全局变量同名，则局部变量屏蔽全局变量。
 - 在函数体内可以直接引用全局变量，但如果函数内任意位置有为该变量赋值的操作，该变量在这个作用域内就是局部变量，除非使用`global`进行了声明。
- ✱ 可以使用内置函数`globals()`和`locals()`查看并输出全局变量和局部变量列表。

```
x = 3
def f():
    print(x)
    x = 5
    print(x)

f()
```

变量赋值将创建局部变量x，则上条语句中引用x出错

`UnboundLocalError: local variable 'x' referenced before assignment`



作用域



■ 示例：在函数内通过关键字global来定义新的全局变量。

✱ 变量在函数外没有定义，在函数内也可用global将一个变量声明为全局变量，然后给该变量赋值，该函数执行后，将增加一个新的全局变量。

```
def demo():  
    y = 4    # 定义局部变量 y  
    global z  
    z = 5    # 定义全局变量 z  
    print('inside: y={},z={}'.format(y,z))
```

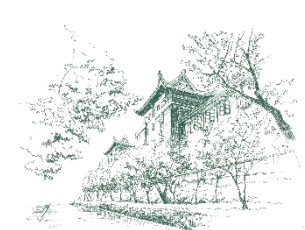
```
demo()  
print('outside: z={}'.format(z))
```

注意，此处z = 5不能缺少，否则报错：

NameError: name 'z' is not defined

程序执行结果如下：

inside: y=4,z=5
outside: z=5



作用域



武汉大学
WUHAN UNIVERSITY

■ 示例：在函数内引用全局变量时可能遇到的问题和解决方法。

```
# 定义全局变量
x = 'outside'
y = 'global'

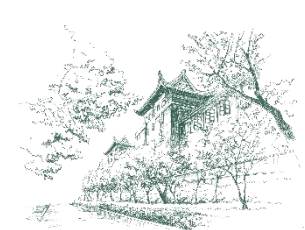
# 定义函数
def f():
    x = 'inside'      # x是局部变量
    print("x is", x)
    print("y is", y)  # y是全局变量

f()
print("x is", x)      # x是全局变量
```

- 在这个程序中，容易出现错误的地方是在函数f()的内部不能直接修改全局变量x的值，因为执行第1条语句x = 'inside'将创建局部变量x，而不是给全局变量x赋值。
- 除了不能直接给全局变量赋值以外，在函数内部没有重名的情况下可以直接引用全局变量，f()函数的最后一条语句print("y is", y)中的变量y就是全局变量y。
- 如果要在函数内部修改全局变量的值，必须先用global进行声明。

程序执行结果如下：

```
x is inside
y is global
x is outside
```



作用域



武汉大学
WUHAN UNIVERSITY

■ 示例：如果要在函数内部修改全局变量的值，必须先用global进行声明。

```
# 定义全局变量  
x = 'outside'  
y = 'global'
```

```
# 定义函数
```

```
def f():
```

```
    global x
```

```
    x = 'inside'
```

```
    print("x is", x)
```

```
    print("y is", y)
```

声明x是全局变量

y是全局变量

```
f()
```

```
print("x is", x)
```

x是全局变量

在函数f中增加了一条声明语句global x，那么函数内的x就是全局变量x，第2条语句执行后就修改了全局变量x所引用的对象，在输出结果中可以看到全局变量x引用'inside'

执行程序得到下面的结果：

```
x is inside  
y is global  
x is inside
```




作用域



武汉大学

WUHAN UNIVERSITY

■ 示例：使用非本地语句nonlocal声明上级函数中定义的局部变量。

定义外部函数

```
def outer_function():  
    tax_rate = 0.17  
    print('外部函数的 tax rate =', tax_rate)
```

定义内部函数

```
def inner_func():  
    nonlocal tax_rate # 声明tax_rate不是本地局部变量，而是上级函数中的局部变量  
    tax_rate = 0.05  
    print('内部函数的 tax_rate =', tax_rate)
```

```
inner_func()  
print('外部函数的 tax_rate =', tax_rate)
```

```
outer_function()
```

与在函数内修改全局变量的值类似，在嵌套函数中，如果要为在上级函数中定义的局部变量赋值，可以使用nonlocal语句，表明变量不是所在函数的本地局部变量，而是其上级函数中的局部变量。nonlocal语句可以指定多个非本地局部变量，例如：

```
nonlocal x, y, z
```

执行程序得到下面的结果：

```
外部函数的 tax rate = 0.17  
内部函数的 tax_rate = 0.05  
外部函数的 tax_rate = 0.05
```



武汉大学
WUHAN UNIVERSITY

06

模块和包



模块



武汉大学

WUHAN UNIVERSITY

■ 在Python中，模块和包是组织和管理代码的重要概念。通过模块和包的使用，可以更好地组织和管理代码，提高代码的可重用性和可维护性。

■ 模块 (Module)

- ✿ 模块是一个包含了Python代码的文件，它可以包含变量、函数、类等定义。
- ✿ 在其他Python程序中，可以使用import语句导入模块，并使用其中定义的内容。
- ✿ 通过将相关的代码放在一个模块中，可以提高代码的可读性和可维护性。



创建和使用模块示例



✿ 创建一个名为 “my_module.py” 的模块

```
# my_module.py

def greet(name):
    print(f"Hello, {name}!")

def add(a, b):
    return a + b
```

➤ 此示例中的my_module.py和my_module_user.py两个程序文件是放在同一个目录中，把模块保存到当前工作目录，可以直接导入并调用其中函数。

➤ 若不在同一目录中，则导入时需要添加相应的路径信息。怎么知道当前工作目录呢？可以使用以下代码：

```
import os
os.getcwd()
```

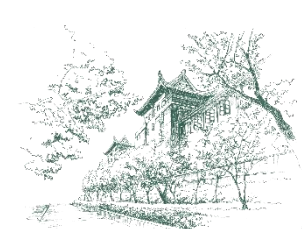
✿ 在另一个Python程序中，使用import语句导入该模块，并使用其中定义的函数

```
# my_module_user.py

import my_module

my_module.greet("Alice")      # 调用模块中的函数
result = my_module.add(3, 5)  # 调用模块中的函数并获取返回值
print(result)
```

程序执行结果如下：
Hello, Alice!
8



包



包 (Package)

- ✱ 包是一个包含了多个模块的目录，它可以帮助更好地组织和管理代码。通过使用包，可以将相关的模块组织在一起，形成一个更大的代码库。
- ✱ 一个包通常包含一个特殊的文件 “__init__.py”，用于标识该目录是一个包。
- ✱ 例如，可以创建一个名为 “my_package” 的包，包含以下文件结构：

```
my_package/  
  __init__.py  
  module1.py  
  module2.py
```

- 其中，“__init__.py” 文件是一个空文件，用于标识 “my_package” 是一个包。
- 在其他Python程序中，可以使用import语句导入包中的模块：

```
import my_package.module1  
import my_package.module2  
  
my_package.module1.function1()  
my_package.module2.function2()
```




再看import语句

import语句

- import语句用于导入模块或包，使得可以在当前的代码中使用其中定义的内容（变量、函数、类或模块等）。

几种不同的import语句形式：

(1) 导入整个模块

import **module_name**

- 这种形式将整个模块导入，并使用“**module_name**”作为模块的名称前缀来访问其中的内容。

(2) 导入模块中的特定内容

from **module_name** **import** **item_name**

- 这种形式从模块中导入指定的“**item_name**”，可以是变量、函数、类等。
- 在当前代码中，可以直接使用“**item_name**”访问导入的内容，无需使用模块名称前缀。



再看import语句



■ 几种不同的import语句形式:

✿ (3) 导入模块中的所有内容

from module_name import *

- 这种形式导入模块中的所有内容。
- 在当前代码中，可以直接使用导入的内容，无需使用模块名称前缀。
- 但是，这种形式可能导致命名冲突和不可预测的行为，因此不推荐在开发正式软件中使用。

✿ (4) 给导入的模块或内容起别名

import module_name as alias_name

from module_name import item_name as alias_name

- 这种形式可以为导入的模块或内容指定一个别名，以方便地使用。
- 使用别名后，可以使用“**alias_name**”来访问导入的模块或内容。



再看import语句



■ 搜索模块

- ✿ 模块的搜索路径是在Python解释器启动时确定的，它包括当前目录、已安装的标准库和其他第三方库。

■ 在导入模块时，Python解释器会按照搜索路径的顺序查找模块文件

✿ (1) 当前目录

- 启动交互式Python的目录，或Python主程序位于的目录。

✿ (2) 操作系统环境变量中指定的目录

✿ (3) Python标准库目录



模块的__name__属性

■ 模块的__name__属性

✿ 每个.py文件在运行时都有一个__name__属性。

➤ 如果作为模块导入，该属性值为模块名。

➤ 如果文件独立运行，该属性值为“__main__”。

✿ 通过对其属性值的判断，使文件在两种情况下都能正确使用。

■ 模块的__name__属性的应用举例

✿ 举例：在mymath.py种添加以下代码。

```
def test_mymath():  
    print(add(12, 34))  
    print(sub(90, 80))  
    print(mul(6, 7))  
    print(div(9, 8))  
  
if __name__ == '__main__':  
    test_mymath()
```



模块的__name__属性

模块的__name__属性的应用举例

✿ 使用方式1：直接运行mymath.py文件，对模块中的函数进行测试

```
(base) D:\Course\pycourse>python mymath.py  
46  
10  
42  
1.125
```

✿ 使用方式2：导入mymath模块，使用其中的函数

```
>>> import mymath  
>>> print(mymath.add(123, 456))  
579
```



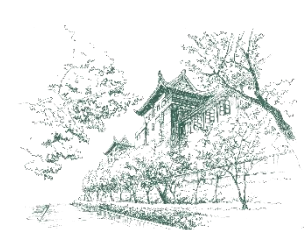

Python标准库



武汉大学
WUHAN UNIVERSITY

Python标准库

- ✱ 随着每个Python版本的发布，会同时发布该版本的标准库。
- ✱ Python的标准库中包含很多模块，为操作系统、解释器和互联网之间的交互提供了丰富的工具。其中既有Python语言自身特定的类型和声明，也包含一些只用于少数程序的模块。所有这些模块都得到充分测试，可以用来作为应用开发的起点。



Python标准库



武汉大学
WUHAN UNIVERSITY

Python标准库中的基础模块

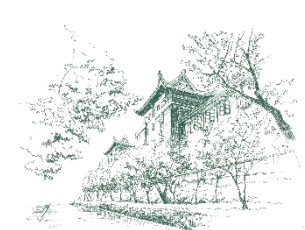
- ✿ 支持内建数据类型操作的模块，如前面提到的用于数学计算操作的`math`模块；为复数提供类似操作的`cmath`模块；以及实现常用字符串处理的`string`模块等。
- ✿ `os`模块：包含了常用的操作系统功能。
- ✿ `sys`模块：通过该模块可以访问程序解释器相关参数，如解释器版本号、模块搜索路径等。
 - 例如：`print(sys.path)`
- ✿ `datetime`和`time`模块：用于处理日期时间。
- ✿ `random`模块：提供了产生随机数（以及随机字符）的多种方法。



武汉大学
WUHAN UNIVERSITY

07

实践指导



用Pandas进行统计分析

Pandas

- ✱ **Pandas是一个快速、强大、灵活且易于使用的数据处理与分析工具，其底层依赖NumPy实现高效存储和计算。**
- ✱ **Pandas提供了两种核心数据结构：**
 - **Series**（带标签的一维数组对象）
 - **DataFrame**（带标签的二维表格对象）
- ✱ **Pandas的主要功能有：**
 - （1）支持多种文件格式的读写，如文本文件、CSV、Excel、JSON和SQL数据库等；
 - （2）数据清洗，如缺失值处理、重复值处理和数据类型转换等；
 - （3）数据操作，如筛选、排序、合并、分组、聚合、映射、数据透视表等；
 - （4）统计分析；
 - （5）数据可视化，基于Matplotlib，提供简单的绘图接口；
 - （6）时间序列分析，提供时间戳和时间差对象，支持时间序列的频率转换。
- ✱ **接下来，让我们了解一下如何创建和访问Series和DataFrame对象。**



创建和访问Series对象

(1) Series

- ✿ Series由标签索引和值两部分组成。
- ✿ 使用pd.Series()构造函数可以创建Series对象，构造函数的基本语法格式如下：
`pd.Series(data=None, index=None)`
- ✿ 其中：
 - data是用于创建Series的数据，支持多种类型，如列表、字典、NumPy数组、另一个Series或标量值等；
 - index用于设置标签索引，支持多种类型，必须与data的长度一致（如果data是字典，那么自动使用字典的键作为标签索引；如果没有明确指定标签索引,那么自动使用从0开始的非负整数作为标签索引）。



创建和访问Series对象



(1) Series

创建Series的例子

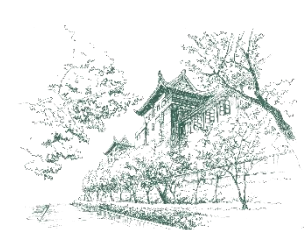
```
>>> import numpy as np
>>> import pandas as pd
```

导入Pandas（需确保已经安装Pandas库）

```
>>> pd.Series([1, 3, 5, np.nan])
0    1.0
1    3.0
2    5.0
3    NaN
dtype: float64
```

根据列表创建Series，未明确指定标签索引

```
>>> pd.Series(np.random.randn(3), index=['x', 'y', 'z']) # 根据NumPy数组创建Series，指定标签索引
x    1.764052
y    0.400157
z    0.978738
dtype: float64
```



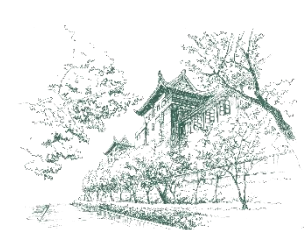
创建和访问Series对象



(1) Series

创建Series的例子

```
>>> s = pd.Series({'b': 10, 'a': 5, 'c': 10, 'd': 20})      # 根据字典创建Series
>>> s
b      10
a       5
c      10
d      20
dtype: int64
```



创建和访问Series对象



武汉大学
WUHAN UNIVERSITY

(1) Series

访问Series的例子

使用loc属性，基于标签索引访问元素

```
>>> print(s.loc['a'])
```

```
5
```

```
>>> s.loc['a'] = -5
```

```
>>> s.loc[['a', 'd']]
```

```
a      -5
```

```
d      20
```

```
dtype: int64
```

```
>>> s.loc['b':'a']
```

```
b      10
```

```
a      -5
```

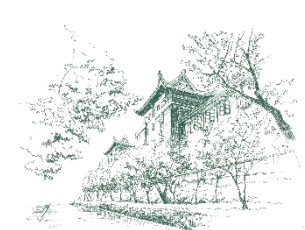
```
dtype: int64
```

访问一个元素

修改元素的值，如果指定的标签索引不存在，则添加元素

列表方式访问多个元素

切片方式访问元素（注意，此处包含终止标签）



创建和访问Series对象



武汉大学
WUHAN UNIVERSITY

(1) Series

✦ 访问Series的例子

使用*iloc*属性, 基于位置索引访问元素

```
>>> print(s.iloc[1])  
-5
```

```
>>> s.iloc[[1, 3]]
```

```
a    -5
```

```
d    20
```

```
dtype: int64
```

```
>>> s.iloc[0:2]
```

```
b    10
```

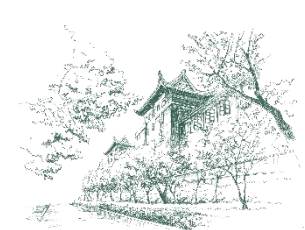
```
a    -5
```

```
dtype: int64
```

访问一个元素

列表方式访问多个元素

切片方式访问元素



创建和访问Series对象



武汉大学
WUHAN UNIVERSITY

(1) Series

访问Series的例子

直接基于标签索引访问元素

```
>>> print(s['a'])
```

```
-5
```

```
>>> s['b':'a']
```

```
b      10
```

```
a      -5
```

```
dtype: int64
```

使用`at`，基于标签索引访问单个元素

```
>>> print(s.at['c'])
```

```
10
```

使用`iat`，基于位置索引访问单个元素

```
>>> print(s.iat[2])
```

```
10
```

访问一个元素

切片方式访问元素（注意，此处包含终止标签）

使用`iat`，通过位置索引访问一个元素



创建和访问Series对象



武汉大学
WUHAN UNIVERSITY

(1) Series

访问Series的例子

```
>>> s[s > 5]
b      10
c      10
d      20
dtype: int64
```

通过布尔索引访问



创建和访问DataFrame对象

(2) DataFrame

- ✱ DataFrame是Pandas中最常用的数据结构，可以看作由若干个Series组成。
- ✱ 使用pd.DataFrame()构造函数可以创建DataFrame对象，构造函数的基本语法格式如下：

DataFrame(data=None, index=None, columns=None)

✱ 其中：

- data是用于创建DataFrame的数据，支持多种类型，如字典、列表、NumPy数组、Series或另一个DataFrame等；
- index用于设置行标签索引；
- columns用于设置列标签索引（即列名）。



创建和访问DataFrame对象



(2) DataFrame

创建DataFrame的例子

```
>>> import numpy as np
>>> import pandas as pd
```

导入Pandas (需确保已经安装Pandas库)

```
>>> pd.DataFrame({'x': [1, 2], 'y': [3, 4], 'z': [5, 6]}) # 根据列表字典创建DataFrame, 没有明确指定行标签索引
```

	x	y	z
0	1	3	5
1	2	4	6

```
>>> d = {
    'one': pd.Series([1.0, 2.0, 1.0], index=['a', 'b', 'c']),
    'two': pd.Series([1.0, 2.0, 3.0, 2.0], index=['a', 'b', 'c', 'd']),
}
```

```
>>> pd.DataFrame(d) # 根据Series字典创建DataFrame
```

	one	two
a	1.0	1.0
b	2.0	2.0
c	1.0	3.0
d	NaN	2.0



创建和访问DataFrame对象

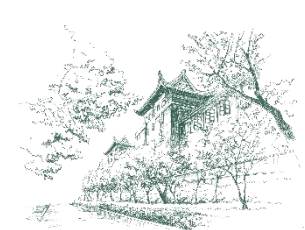


(2) DataFrame

创建DataFrame的例子

```
>>> dates = pd.date_range('20250101', periods=6, freq='MS') # 生成日期序列
>>> pd.DataFrame(np.random.randn(6,4), index=dates, columns=list('ABCD')) # 根据NumPy数组创建DataFrame
```

	A	B	C	D
2025-01-01	0.636350	0.440902	2.339545	0.139011
2025-02-01	1.024214	-0.353824	0.134863	-1.712178
2025-03-01	-0.484195	-0.321369	-0.763129	-0.597539
2025-04-01	-0.211402	0.637620	1.329028	0.393731
2025-05-01	-0.741332	0.873069	-0.016883	-0.979553
2025-06-01	-0.080006	-1.621851	-0.746100	0.118736



创建和访问DataFrame对象



🖥️ (2) DataFrame

🌟 创建DataFrame的例子

```
>>> d = {  
    'Name': ['张三', '李四', '王五', '赵六'],  
    'Age': [22, 20, 20, 19],  
    'Gender': ['男', '女', '男', '男'],  
    'Mobile': ['12345678901', '12345678902', '12345678903', '12345678904']  
}
```

```
>>> index = ['first', 'second', 'third', 'fourth']
```

```
>>> df = pd.DataFrame(d, index=index) # 根据列表字典创建DataFrame
```

```
>>> df
```

	Name	Age	Gender	Mobile
first	张三	22	男	12345678901
second	李四	20	女	12345678902
third	王五	20	男	12345678903
fourth	赵六	19	男	12345678904



创建和访问DataFrame对象



武汉大学
WUHAN UNIVERSITY

❏ (2) DataFrame

✦ 访问DataFrame的例子

使用loc属性，基于行标签索引和列标签索引访问元素

```
>>> df.loc['first', 'Name']  
'张三'
```

```
>>> df.loc[['first', 'third'], ['Name', 'Mobile']]
```

	Name	Mobile
first	张三	12345678901
third	王五	12345678903

```
>>> df.loc['first']
```

Name	张三
Age	22
Gender	男
Mobile	12345678901

```
Name: first, dtype: object
```

访问一个元素

列表方式访问多个元素

访问单行，返回Series



创建和访问DataFrame对象



📊 (2) DataFrame

🌟 访问DataFrame的例子

```
>>> df.loc['first': 'third']
```

	Name	Age	Gender	Mobile
first	张三	22	男	12345678901
second	李四	20	女	12345678902
third	王五	20	男	12345678903

```
>>> df.loc[:, 'Name']
```

first	张三
second	李四
third	王五
fourth	赵六

```
Name: Name, dtype: object
```

```
>>> df.loc[:, 'Name': 'Gender']
```

	Name	Age	Gender
first	张三	22	男
second	李四	20	女
third	王五	20	男
fourth	赵六	19	男

切片方式访问多行（注意，此处包含终止标签）

访问单列，返回Series

切片方式访问多列（注意，此处包含终止标签）



创建和访问DataFrame对象



🖥️ (2) DataFrame

🌟 访问DataFrame的例子

使用*iloc*属性, 基于行位置索引和列位置索引访问元素

```
>>> df.iloc[0, 0]
```

```
'张三'
```

```
>>> df.iloc[[0, 2], [0, 3]]
```

	Name	Mobile
first	张三	12345678901
third	王五	12345678903

```
>>> df.iloc[0]
```

Name	张三
Age	22
Gender	男
Mobile	12345678901

```
Name: first, dtype: object
```

```
>>> df.iloc[0: 2]
```

	Name	Age	Gender	Mobile
first	张三	22	男	12345678901
second	李四	20	女	12345678902

访问一个元素

列表方式访问多个元素

访问单行, 返回Series

切片方式访问多行



创建和访问DataFrame对象



武汉大学
WUHAN UNIVERSITY

🖥️ (2) DataFrame

🌟 访问DataFrame的例子

使用*iloc*属性, 基于行位置索引和列位置索引访问元素

```
>>> df.iloc[:, 0]
```

```
first      张三  
second     李四  
third      王五  
fourth     赵六
```

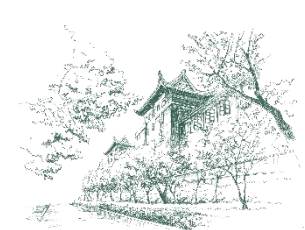
```
Name: Name, dtype: object
```

```
>>> df.iloc[:, 0: 2]
```

	Name	Age
first	张三	22
second	李四	20
third	王五	20
fourth	赵六	19

访问单列, 返回Series

切片方式访问多列



创建和访问DataFrame对象



🖥️ (2) DataFrame

🌟 访问DataFrame的例子

直接基于列标签索引访问列

```
>>> df['Name']
```

```
first      张三  
second     李四  
third      王五  
fourth     赵六
```

```
Name: Name, dtype: object
```

```
>>> df[['Name', 'Age']]
```

	Name	Age
first	张三	22
second	李四	20
third	王五	20
fourth	赵六	19

访问单列，返回Series

访问多列



创建和访问DataFrame对象



🖥️ (2) DataFrame

🌟 访问DataFrame的例子

直接基于行位置索引访问行

```
>>> df[0:2]
```

	Name	Age	Gender	Mobile
first	张三	22	男	12345678901
second	李四	20	女	12345678902

切片方式访问多行

使用at，基于行标签索引和列标签索引访问单个元素

```
>>> df.at['first', 'Name']
```

```
'张三'
```

使用iat，基于行位置索引和列位置索引访问单个元素

```
>>> df.iat[0, 0]
```

```
'张三'
```




创建和访问DataFrame对象

🖥️ (2) DataFrame

🌟 访问DataFrame的例子

通过布尔索引访问

```
>>> df[df['Age'] >= 20]
```

	Name	Age	Gender	Mobile
first	张三	22	男	12345678901
second	李四	20	女	12345678902
third	王五	20	男	12345678903

```
>>> df[df['Gender'] == '男']
```

	Name	Age	Gender	Mobile
first	张三	22	男	12345678901
third	王五	20	男	12345678903
fourth	赵六	19	男	12345678904

注意：此处必须使用按位与运算符&，不能使用逻辑与运算符and

```
>>> df[(df['Age'] >= 20) & (df['Gender'] == '男')]
```

	Name	Age	Gender	Mobile
first	张三	22	男	12345678901
third	王五	20	男	12345678903



创建和访问DataFrame对象

(2) DataFrame

访问DataFrame的例子

添加列，如果该列标签索引已经存在，则覆盖

```
>>> df['Province'] = ['湖北', '海南', '湖南', '广东']
```

```
>>> df
```

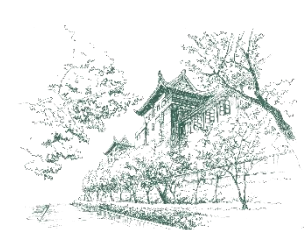
	Name	Age	Gender	Mobile	Province
first	张三	22	男	12345678901	湖北
second	李四	20	女	12345678902	海南
third	王五	20	男	12345678903	湖南
fourth	赵六	19	男	12345678904	广东

添加行，如果该行标签索引已经存在，则覆盖

```
>>> df.loc['five'] = ['李华', 19, '女', '12345678922', '湖北']
```

```
>>> df
```

	Name	Age	Gender	Mobile	Province
first	张三	22	男	12345678901	湖北
second	李四	20	女	12345678902	海南
third	王五	20	男	12345678903	湖南
fourth	赵六	19	男	12345678904	广东
five	李华	19	女	12345678922	湖北



应用案例1：用Pandas进行统计分析



■ 编写利用Pandas实现学生成绩统计分析任务的程序。

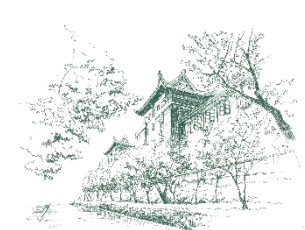
✱ 测试数据：有5位学生3门课程的考试成绩。

➤ 用列表来保存3门课程；

➤ 用嵌套的列表来保存学生成绩。

- 为了便于引用每个学生的成绩，可以用嵌套的列表来保存由这些成绩构成的成绩表。
- 每位学生的3门课程成绩保存在一个列表中，然后把5个结构相同的学生课程成绩列表作为元素保存在一个大列表中。

```
course_names = ['Python', '数据分析', '机器学习']  
score_sheet = [  
    [85, 87, 80],  
    [93, 91, 88],  
    [88, 90, 87],  
    [82, 78, 80],  
    [91, 88, 85]  
]
```



应用案例1：用Pandas进行统计分析



武汉大学
WUHAN UNIVERSITY

■ 编写利用Pandas实现学生成绩统计分析任务的程序。

✿ 函数功能：

- (1) `plot_course_scores()`函数的功能是绘制课程成绩箱形图。
 - 参数`df`是包含各门课程成绩的`DataFrame`;
 - `figsize`设置图表尺寸，默认为(8, 4);
- (2) `calculate_and_display_statistics()`函数的功能是计算并格式化输出统计指标。
 - 参数`df`是包含各门课程成绩的`DataFrame`。



应用案例1：用Pandas进行统计分析



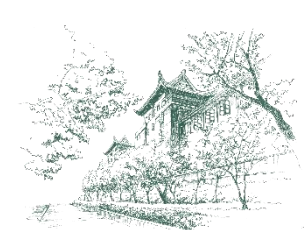
■ 编写利用Pandas实现学生成绩统计分析任务的程序。

✿ 代码实现：

```
import pandas as pd
import matplotlib.pyplot as plt

def plot_course_scores(df, figsize=(10, 6)):
    """绘制课程成绩箱形图"""
    plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体
    df.boxplot(figsize=figsize, ylabel='分数') # 绘制箱形图
    plt.title('各门课程的成绩分布情况')
    plt.show()

def calculate_and_display_statistics(df):
    """计算并输出课程成绩统计指标"""
    stat = pd.DataFrame() # 创建空DataFrame
    stat['平均值'] = df.mean() # 添加平均值列
    stat['中位数'] = df.median() # 添加中位数列
    stat['众数'] = df.mode().iloc[0] # 添加众数列（众数可能有多个，只取第一个）
    stat['方差'] = df.var() # 添加方差列
    stat['标准差'] = df.std() # 添加标准差列
```



应用案例1：用Pandas进行统计分析



武汉大学
WUHAN UNIVERSITY

■ 编写利用Pandas实现学生成绩统计分析任务的程序。

✿ 代码实现：

```
# 重置索引，将课程名称作为一列
stat.reset_index(inplace=True)
stat = stat.rename(columns={'index': '课程'})

# 设置显示格式，解决对齐问题
pd.set_option('display.unicode.ambiguous_as_wide', True)
pd.set_option('display.unicode.east_asian_width', True)

# 格式化输出stat，去掉行索引，浮点数保留1位小数
print(stat.to_string(index=False, float_format='%.1f'))

return stat

def main():

# 创建测试数据
course_names = ['Python', '数据分析', '机器学习']
```




应用案例1：用Pandas进行统计分析

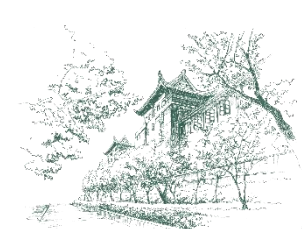


武汉大学
WUHAN UNIVERSITY

■ 编写利用Pandas实现学生成绩统计分析任务的程序。

✿ 代码实现：

```
score_sheet = [  
    [85, 87, 80],  
    [93, 91, 88],  
    [88, 90, 87],  
    [82, 78, 80],  
    [91, 88, 85]  
]  
  
# 根据score_sheet创建DataFrame, columns参数指定列名  
df = pd.DataFrame(score_sheet, columns=course_names)  
  
# 调用绘图函数  
plot_course_scores(df)  
  
print('统计结果如下：')  
# 调用统计函数  
calculate_and_display_statistics(df)  
  
if __name__ == "__main__":  
    main()
```



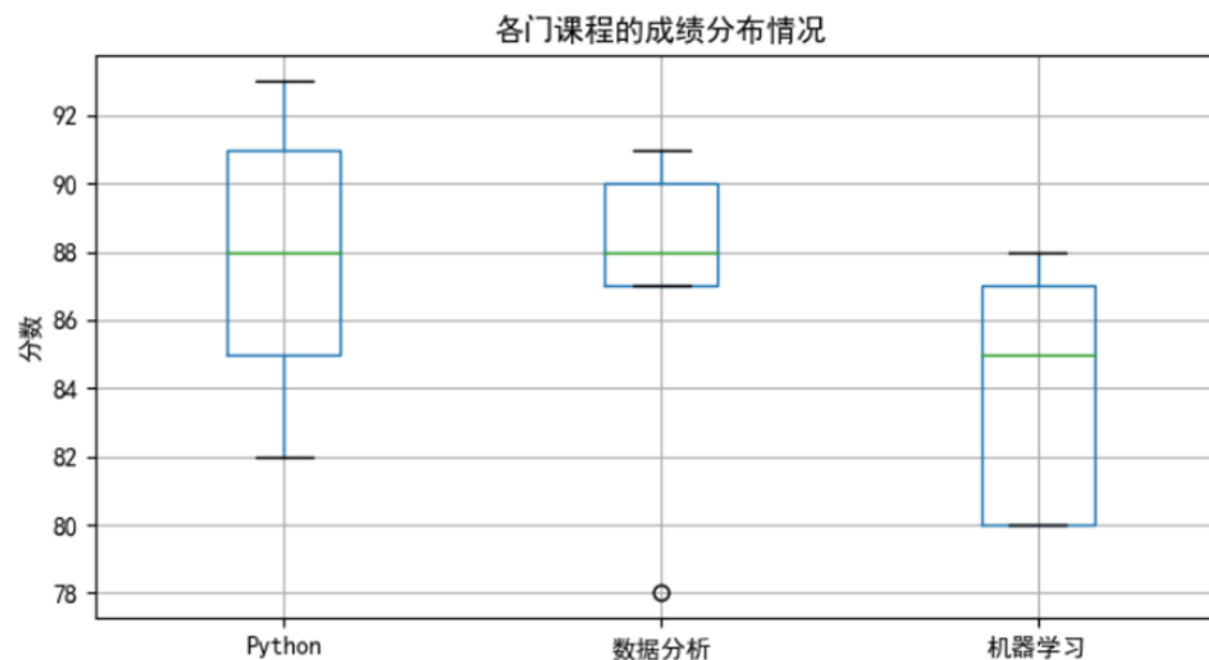
应用案例1：用Pandas进行统计分析



武汉大学
WUHAN UNIVERSITY

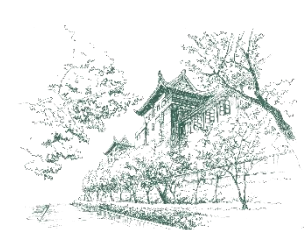
■ 编写利用Pandas实现学生成绩统计分析任务的程序。

✿ 程序执行结果如下：



统计结果如下：

课程	平均值	中位数	众数	方差	标准差
Python	87.8	88.0	82.0	19.7	4.4
数据分析	86.8	88.0	78.0	26.7	5.2
机器学习	84.0	85.0	80.0	14.5	3.8



小结



■ 了解函数式编程的概念。

- ✿ 理解和掌握lambda表达式（匿名函数）。
- ✿ 掌握内置函数map()、内置函数filter()的用法。
- ✿ 掌握functools模块的reduce()函数的用法。

■ 了解和掌握递归函数的定义和使用。

■ Python允许嵌套定义函数，即在一个函数内部定义另外一个函数。

■ 定义函数时，函数前面可以添加一个或多个装饰器，以增强函数功能。

■ 理解命名空间和作用域的基本概念

- ✿ 掌握局部变量和全局变量的使用。
- ✿ 掌握global、非本地语句nonlocal的使用。



小结



■ 通过模块和包来组织和管理程序代码。用模块化方法设计实现复杂程序。

- ✱ 模块是一个包含了Python代码的文件，使用import语句导入模块，并使用其中定义的内容。
- ✱ 包是一个包含了多个模块的目录，“__init__.py”用于标识该目录是一个包。包可以帮助更好地组织和管理代码。通过使用包，可以将相关的模块组织在一起，形成一个更大的代码库。
- ✱ 掌握几种不同的import语句形式。

■ Pandas是一个快速、强大、灵活且易于使用的数据处理与分析工具。

- ✱ 其底层依赖NumPy实现高效存储和计算。
- ✱ Pandas提供了两种核心数据结构：
 - Series（带标签的一维数组对象）
 - DataFrame（带标签的二维表格对象）



武汉大学

WUHAN UNIVERSITY

谢谢

THANK YOU

